

Windows Security Model Simulation

Access Tokens, Discretionary Access Control Lists & Privilege Checks



Subject: Operating Systems

Topic: Windows Access Control Architecture

Implementation Language: C (Standard C11)

GitHub Repository: <https://github.com/Sriram-Nambiar/os-security-simulator>

Table of Contents

1. [Introduction](#)
2. [Objectives](#)
3. [Theoretical Background](#)
4. [Security Architecture & Relationships](#)
5. [DACL Evaluation Algorithm & Flowchart](#)
6. [Implementation Architecture & Methodology](#)
7. [Program Execution & Output Screenshots](#)
8. [Comprehensive Test Scenario Analysis](#)
9. [Compilation & Repository Information](#)
10. [Conclusion & Key Takeaways](#)

1. Introduction

Resource protection and access authorization are fundamental responsibilities of modern operating systems. Microsoft Windows NT implements a robust **Discretionary Access Control (DAC)** security subsystem. Under this model, every process runs within the security boundary of an *Access Token* representing the caller's credentials, and every securable resource (such as files, directories, registry keys, and named pipes) is protected by a *Security Descriptor* holding a *Discretionary Access Control List (DACL)*.

This project simulates this complete authorization mechanism in a high-fidelity software model. It demonstrates how security identifiers (SIDs), access masks, ordered access control entries (ACEs), and administrative privileges interact to produce an authorization outcome: **ACCESS GRANTED** or **ACCESS DENIED**.

Key Takeaway: The project focuses on the core conceptual algorithm used by the Windows Security Reference Monitor (SRM), demonstrating why ACE evaluation order is critical and how explicit denials safeguard system security.

2. Objectives

1. **Model Security Principals:** Construct simulated User and Group accounts identified by Security Identifiers (SIDs).
2. **Implement Access Tokens:** Attach an access token to each process containing user identity, group memberships, and system privileges.
3. **Structure Security Descriptors & DACLs:** Protect resources using ordered lists of Access Control Entries (ACEs).
4. **Implement Sequential Evaluation:** Faithfully simulate the Windows DACL walk, emphasizing *Owner Authority*, *Explicit Deny Precedence*, *Allow Accumulation*, and *Implicit Deny*.
5. **Demonstrate Privilege Override:** Illustrate how administrative capabilities like `SeBackupPrivilege` bypass normal DACL restrictions for authorized system tools.
6. **Provide Dual Operation Modes:** Support both an automated 8-scenario demonstration and a menu-driven interactive mode.

3. Theoretical Background

3.1 The Access Token

When a user logs into a Windows system, the Local Security Authority Subsystem Service (LSASS) authenticates the credentials and generates an **Access Token**. Every process launched by or on behalf of that user inherits a copy of this token.

Field	Function in Windows	Representation in Simulator
User SID	Unique binary identifier of the user account	User identifier (UID) associated with an account name
Group SIDs	List of all security groups the user belongs to	Array of group identifiers inherited at process creation
Privileges	System-wide administrative rights assigned to the token	Named capability flags (e.g., <code>SeBackupPrivilege</code>)

3.2 Security Descriptors & DACLs

Every securable object maintains a Security Descriptor. The most important component for authorization is the **Discretionary Access Control List (DACL)**, created and maintained by the resource owner. A DACL contains an ordered sequence of **Access Control Entries (ACEs)**.

ACE Element	Description
ACE Type	Specifies whether access is permitted (<code>ACCESS_ALLOWED_ACE_TYPE</code>) or prohibited (<code>ACCESS_DENIED_ACE_TYPE</code>).
Trustee SID	Identifies the specific user or group to which this rule applies.
Access Mask	A 32-bit integer where individual bits represent specific permissions (Read, Write, Execute, Delete).

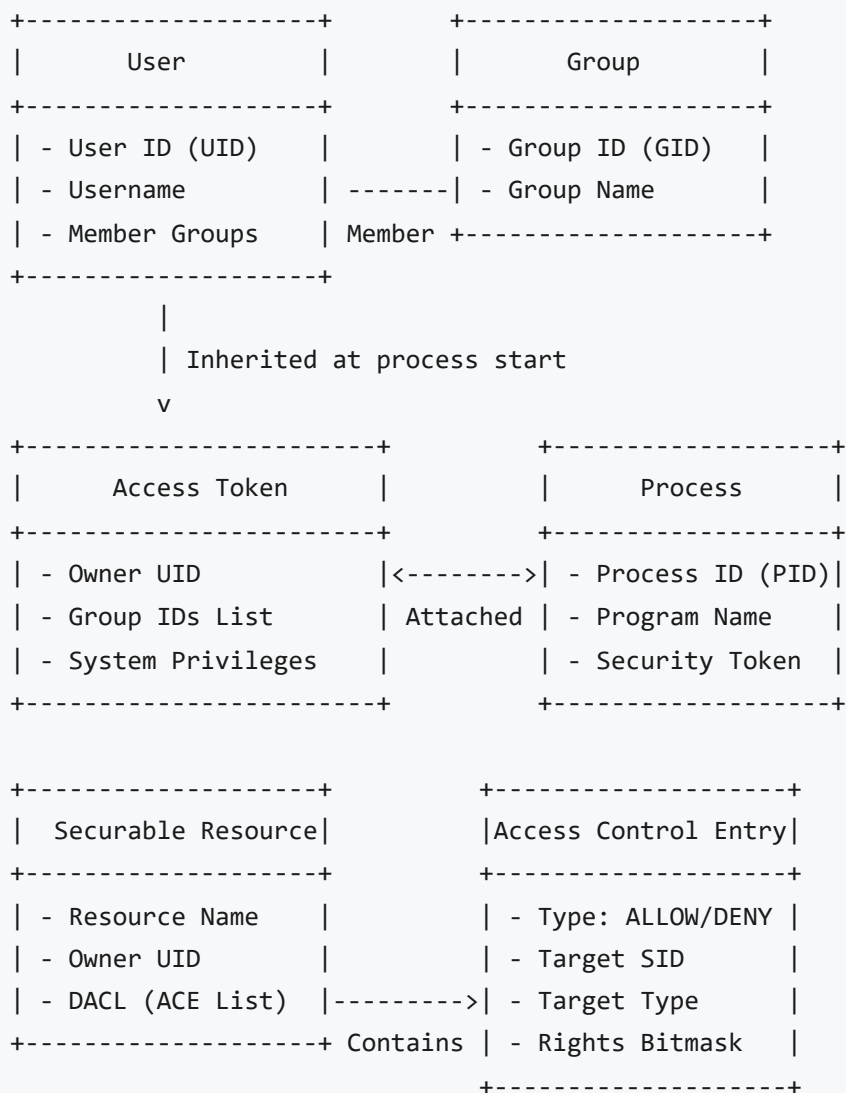
3.3 Why Order Matters: The Deny-Before-Allow Principle

Unlike simple permission models that only check for allowances, Windows evaluates ACEs in strict sequential order from top to bottom. Because an explicit **DENY** matching any requested permission causes immediate rejection, Windows standard practice places all DENY entries ahead of ALLOW entries in the canonical DACL order. This ensures that restricted memberships cannot be overridden by broad group allowances.

4. Security Architecture & Relationships

4.1 Entity Relationship Diagram

The following diagram illustrates how security principals, processes, tokens, and resources connect:



4.2 Permission Bitmask Mapping

The simulator uses standard bit flags representing file system permissions:

Permission	Bit Position	Value	Meaning
READ	Bit 0	0x01	Permission to view file contents or read resource data.
WRITE	Bit 1	0x02	Permission to modify file contents or write data.
EXECUTE	Bit 2	0x04	Permission to execute an application or binary.
DELETE	Bit 3	0x08	Permission to remove the resource from the system.
FULL CONTROL	Bits 0-3	0x0F	All permissions combined (Read + Write + Execute + Delete).

4.3 Simulated Environment Configuration

The simulator initializes a representative corporate environment:

- **Users:** Alice (Administrator), Bob (Standard User), Charlie (Guest), Dave (Backup Technician).
- **Groups:** Administrators , Users , Backup Operators , Guests .
- **Protected Resources:**
 - C:\Confidential\report.docx : Owned by Alice. Denies Guests; allows Administrators (Full) and Users (Read).
 - D:\Shared\project.xlsx : Owned by Bob. Denies Backup Operators (Write/Delete); allows Users (Read/Write).
 - C:\System\boot.ini : Owned by Alice. Allows Administrators (Read); all others implicitly denied.

5. DACL Evaluation Algorithm & Flowchart

5.1 Step-by-Step Authorization Logic

When an application requests access to a resource, the Security Reference Monitor evaluates the request through four sequential phases:

Phase 1: Owner Authority Check

If the process's token UID matches the resource's owner UID, access is immediately **GRANTED** under implicit Full Control.

Phase 2: Privilege Bypass Verification

If the operation qualifies for a system privilege held by the token (e.g., `SeBackupPrivilege` for read operations), the DACL check is bypassed and access is immediately **GRANTED**.

Phase 3: Ordered DACL Traversal

The system scans ACE entries in sequential order (from index 1 to N):

- If the ACE does not match the token's User SID or any Group SID, it is skipped.
- If a matching **DENY** ACE covers *any* of the requested permissions, evaluation terminates and access is **DENIED**.
- If a matching **ALLOW** ACE is found, its permissions are accumulated into the granted rights mask.

Phase 4: Accumulation & Implicit Deny

If all requested permissions have been accumulated, access is **GRANTED**. If the end of the DACL is reached without fulfilling all requested rights, access is **DENIED** (implicit deny by default).

5.2 Decision Flowchart

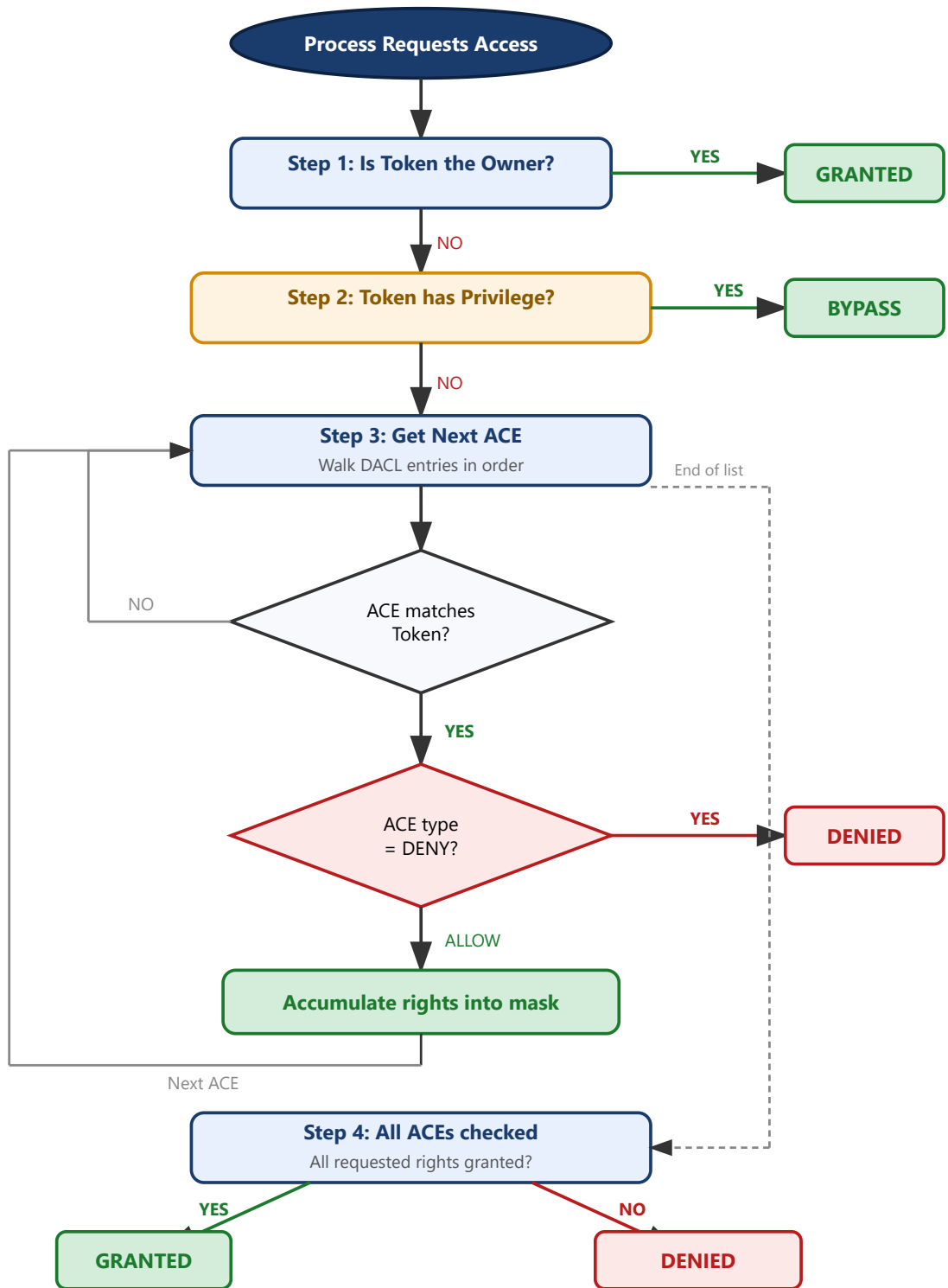


Figure 1: Windows DACL Access Check Decision Pipeline

6. Implementation Architecture & Methodology

6.1 Functional Decomposition

The simulator is structured into distinct, modular functional layers:

Module / Function	Architectural Responsibility
Security Context (Tokens)	Manages process identities, assigning user credentials, multi-group memberships, and special administrative capabilities.
Resource Protection (DACLS)	Encapsulates securable entities with owner metadata and an array of ordered Access Control Entries.
Matching Engine (<code>ace_matches_token</code>)	Verifies whether a rule targets the caller directly by User SID or transitively through any group SIDs present in the token.
Authorization Evaluator (<code>access_check</code>)	Executes the 4-phase evaluation algorithm: owner override, sequential ACE walk, deny enforcement, and rights accumulation.
Privilege Handler (<code>token_has_privilege</code>)	Bypasses standard DACL evaluation when specific administrative capabilities (like <code>SeBackupPrivilege</code>) are asserted.
Console Safety Layer (<code>read_int_safe</code>)	Uses line-buffered stream reading (<code>fgets</code> + <code>sscanf</code>) to guarantee immunity from infinite loops and buffer corruption on arbitrary user input.

6.2 Operational Execution Flow

The program offers two operational modes:

- **Automated Demonstration Mode:** Automatically loads preconfigured users, resources, and processes, and steps through 8 real-world authorization scenarios, printing the evaluation trace and verdict for each.
- **Interactive Exploration Mode:** Provides a live console environment where administrators can view loaded security descriptors, add dynamic ACEs to resources, and evaluate custom access queries in real time.

7. Program Execution & Output Screenshots

7.1 Program Startup & Interactive Mode Menu

Upon launching the simulator, the operator can select between the automated preset test suite or the interactive console. The interactive menu allows real-time inspection and runtime manipulation of security objects.

```
+-----+
| WINDOWS SECURITY MODEL SIMULATION |
| Access Tokens, DACLS & Privilege Checks |
+-----+

1. Run PRESET DEMO (recommended -- 8 scenarios)
2. INTERACTIVE mode (inspect/modify then test)
Enter choice (1 or 2): 2

-----

INTERACTIVE MODE MENU
-----

1. List all users
2. List all resources & DACLS
3. List all processes & tokens
4. Perform an access check
5. Add a new ACE to a resource's DACL
6. Return to main menu
Enter choice (1-6): █
```

Figure 2: Simulator startup interface displaying mode selection and the interactive menu

7.2 Simulated Users, Resources, and DACL Definitions

The simulator presents the loaded security environment. In resource [report.docx](#), the explicit DENY entry for Guests is positioned first, illustrating the canonical deny-before-allow ordering required by Windows.

```

USERS:
  UID 1  Alice      Groups: Administrators, Users
  UID 2  Bob        Groups: Users
  UID 3  Charlie    Groups: Guests
  UID 4  Dave       Groups: Users, Backup Operators

RESOURCES:
Resource : "C:\Confidential\report.docx"  Owner: Alice
DACL (3 ACEs):
  ACE #1: DENY   Guests      [READ WRITE EXECUTE DELETE]
  ACE #2: ALLOW  Administrators [READ WRITE EXECUTE DELETE]
  ACE #3: ALLOW  Users       [READ]

Resource : "D:\Shared\project.xlsx"  Owner: Bob
DACL (2 ACEs):
  ACE #1: DENY   Backup Operators [WRITE DELETE]
  ACE #2: ALLOW  Users            [READ WRITE]

```

Figure 3: Security principals with group memberships and resource DACLs with ordered ACEs

7.3 Access Check Scenario Execution

During scenario evaluation, the simulator prints a detailed reasoning trace showing how the token's credentials were matched against the resource's DACL, culminating in a clear authorization verdict.

ACCESS CHECK SCENARIOS

```

SCENARIO 1: Alice (Admin) reads report.docx
  Process : PID 1 (cmd.exe)
  Resource : C:\Confidential\report.docx
  Requested: [READ]
  >> GRANTED -- requestor is the OWNER (implicit Full Control)
  [GRANTED] RESULT: ACCESS GRANTED

```

```

SCENARIO 2: Bob (User) reads report.docx
  Process : PID 2 (notepad.exe)
  Resource : C:\Confidential\report.docx
  Requested: [READ]
  >> GRANTED -- sufficient ALLOW ACEs found in DACL
  [GRANTED] RESULT: ACCESS GRANTED

```

Figure 4: Detailed reasoning trace for authorization decisions (Owner override and group allowances)

8. Comprehensive Test Scenario Analysis

The simulator evaluates 8 representative scenarios, each validating a distinct principle of the Windows security architecture:

#	Scenario Description	Requested	Result	Underlying Security Principle Demonstrated
1	Alice (Admin) reads <code>report.docx</code>	READ	GRANTED	Owner Authority: Because Alice owns the resource, access is granted immediately without evaluating DACL entries.
2	Bob (User) reads <code>report.docx</code>	READ	GRANTED	Group Allowance: ACE #3 grants READ to the Users group; Bob's token carries this group SID.
3	Bob (User) writes <code>report.docx</code>	WRITE	DENIED	Implicit Deny: No ACE grants WRITE to Users; unmatched rights default to denial.
4	Charlie (Guest) reads <code>report.docx</code>	READ	DENIED	Explicit Deny Precedence: ACE #1 explicitly denies Guests; the denial matches first and halts evaluation immediately.
5	Dave writes <code>project.xlsx</code>	WRITE	DENIED	Deny-Before-Allow Ordering: Dave belongs to both Users (allowed) and Backup Operators (denied). The DENY ACE evaluates first, blocking write access.
6	Dave reads <code>project.xlsx</code>	READ	GRANTED	Targeted Rights Scoping: The DENY ACE only restricts WRITE and DELETE; READ is uninhibited and granted via the Users group.
7	Bob (User) reads <code>boot.ini</code>	READ	DENIED	Administrative Isolation: The DACL only permits Administrators; Bob's token holds no matching allow entries.
8	Dave reads <code>boot.ini</code>	READ	GRANTED	Privilege Override: Dave's token holds <code>SeBackupPrivilege</code> , which bypasses DACL evaluation for backup operations.

9. Compilation & Repository Information

9.1 Prerequisites

The program is written in standard ANSI C (C11) with zero external dependencies and compiles with any standard C compiler:

- **Windows:** MinGW GCC, MSVC, or Clang
- **Linux / macOS:** GCC or Clang

9.2 Compilation Command

Compile the simulator using GCC with warnings enabled:

```
gcc -o windows_acl.exe windows_acl.c -Wall
```

9.3 Execution Command

Run the executable directly in your terminal:

```
.\windows_acl.exe
```

9.4 Source Code & Project Repository

The complete project source code, documentation, and assets are publicly hosted on GitHub:

 **Project Repository:** <https://github.com/Sriram-Nambiar/os-security-simulator>

10. Conclusion & Key Takeaways

Through this simulation, the operational mechanics of the Windows NT security architecture have been explored and verified. The project yields five core architectural conclusions:

1. **Identity Encapsulation:** Access Tokens cleanly separate process authorization from execution logic by packaging User SIDs, Group SIDs, and Privileges into a single transferable credential structure.
2. **Declarative Security:** DACLs enable fine-grained access control on individual objects without requiring hardcoded application logic.
3. **Precedence of Explicit Denials:** Deny-before-allow ordering prevents security regressions when users accumulate multiple group memberships across large organizations.
4. **Secure by Default:** The implicit deny mechanism ensures that unmentioned or ambiguous permissions are rejected automatically without administrative intervention.
5. **Privilege Segregation:** Administrative privileges provide a controlled, auditable mechanism for authorized system services to perform critical maintenance without permanently altering object DACLs.